

Regime-Adaptive Algorithmic Trading: A Hybrid GARCH-LSTM and XGBoost Gatekeeping Framework

Alexandru Lorintz

Faculty of Mathematics and Computer Science

Babeş-Bolyai University

Cluj-Napoca, Romania

alexandru.lorintz@stud.ubbcluj.ro

Abstract—Financial markets exhibit non-stationary behaviors characterized by volatility clustering and abrupt regime shifts. Traditional linear econometric models often fail to capture complex non-linear dependencies during stress events. This paper proposes a modular algorithmic trading framework consisting of two core components: (1) A Hybrid Volatility Model that combines GARCH linearity with Long Short-Term Memory (LSTM) networks to model residual variance errors, and (2) A Momentum Gatekeeper utilizing XGBoost to filter trade entries based on multi-factor probabilities. We conduct a rigorous "Design of Experiments" utilizing high-cap equities (SPY, AAPL, NVDA) and Bayesian hyperparameter optimization. Experimental results on out-of-sample data (2025-2026) demonstrate that the Momentum-XGBoost strategy achieves a Sharpe Ratio > 1.6 on AAPL by effectively reducing false positive signals, while the Volatility Targeting mechanism significantly mitigates maximum drawdown in high-beta assets like TSLA and GOOG compared to a Buy-and-Hold benchmark.

I. INTRODUCTION

Financial time series prediction remains a challenging domain due to stochastic volatility, heavy tails, and concept drift. While Generalized Autoregressive Conditional Heteroskedasticity (GARCH) models are the standard for volatility forecasting, they assume a strict linear dependence on past variance. Conversely, Deep Learning models like LSTMs excel at non-linear pattern recognition but often suffer from overfitting when applied to raw price data.

This study introduces a **Hybrid Residual Learning Framework**. We posit that market volatility is composed of a linear inertial component (explainable by GARCH) and a non-linear residual component (explainable by LSTM). Furthermore, we address the "profitability vs. accuracy" dilemma by implementing a secondary ML layer—an XGBoost Gatekeeper—that acts not as a price predictor, but as a trade filter.

The contributions of this work are:

- A Hybrid **GARCH-LSTM** architecture that predicts volatility residuals to correct econometric underestimations.
- A **Directional Gatekeeper** using XGBoost to classify the probability of sustained momentum, filtering false breakouts.

- A robust backtesting engine with lookahead-bias-free signal generation and transaction cost modeling.

II. METHODOLOGY

This section details the theoretical underpinnings and practical implementation of the three primary models used: the Garman-Klass estimator for data preprocessing, the Hybrid GARCH-LSTM for volatility forecasting, and XGBoost for signal filtering.

A. Data Preprocessing: Garman-Klass Volatility

Before modeling, we must accurately measure historical volatility. Standard "Close-to-Close" estimators ignore price fluctuations that occur *during* the trading day. To capture this intra-day noise, we utilize the **Garman-Klass Estimator**.

This estimator incorporates the Open (O), High (H), Low (L), and Close (C) prices. It is approximately eight times more efficient than simple close-to-close variance estimators because it accounts for the trading range (High minus Low) and the opening jump (Open minus Previous Close).

$$\sigma_{GK}^2 = 0.5 \ln \left(\frac{H_t}{L_t} \right)^2 - (2 \ln 2 - 1) \ln \left(\frac{C_t}{O_t} \right)^2 \quad (1)$$

By using σ_{GK}^2 , we provide our models with a "cleaner" target variable that better represents the true market risk of a given day.

B. Hybrid Volatility Model (GARCH-LSTM)

Volatility (risk) is not constant; it clusters. Turbulent days are likely to be followed by turbulent days. To model this, we use a two-stage hybrid approach.

1) *Stage 1: Linear Modeling with GARCH*: The **GARCH** (Generalized Autoregressive Conditional Heteroskedasticity) model is an econometric tool designed to model time-varying variance. It assumes that current variance depends on:

- **Past Variance terms (GARCH, β)**: The memory of previous volatility.
- **Past Squared Errors (ARCH, α)**: The magnitude of recent shocks (returns).

We fit a GARCH(p, q) model to the log-returns. The conditional variance h_t is forecasted as:

$$h_t = \omega + \sum_{i=1}^p \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^q \beta_j h_{t-j} \quad (2)$$

This model captures the "linear" aspect of market risk—the predictable inertial movements. However, GARCH often reacts slowly to sudden crashes ("black swan" events).

2) *Stage 2: Non-Linear Correction with LSTM*: To fix the slow reaction time of GARCH, we employ a **Long Short-Term Memory (LSTM)** network. LSTMs are a type of Recurrent Neural Network (RNN) designed to learn long-term dependencies in sequence data. Unlike standard RNNs, LSTMs possess internal "gates" (input, forget, and output gates) that regulate the flow of information, preventing the "vanishing gradient" problem.

In our framework, the LSTM does not predict price. Instead, it predicts the **residual error** of the GARCH model. We define the error e_t as:

$$e_t = \sigma_{GK,t} - \hat{\sigma}_{GARCH,t} \quad (3)$$

If e_t is positive, GARCH underestimated the risk. If negative, it overestimated. The LSTM takes technical indicators (Volume, Returns) as input (X_t) and learns to predict this error $\hat{e}_{t+1}$.

The final Hybrid forecast combines the stable linear baseline with the agile non-linear correction:

$$\hat{\sigma}_{Hybrid,t+1} = \hat{\sigma}_{GARCH,t+1} + \text{LSTM}(X_{t-k:t}) \quad (4)$$

C. Momentum Gatekeeper: XGBoost

While volatility models help manage risk/position sizing, they do not generate buy signals. For this, we use a momentum strategy filtered by **XGBoost**.

1) *The Concept: XGBoost*: **XGBoost** (eXtreme Gradient Boosting) is an ensemble learning method based on decision trees. Instead of building one large tree (which might overfit), it builds hundreds of small, weak trees sequentially. Each new tree attempts to correct the errors made by the previous trees. It minimizes a regularized objective function, making it highly effective for structured data.

2) *The Gatekeeper Logic*: We do not ask XGBoost to predict "Will the price go up?". That is too noisy. Instead, we use it as a **Meta-Labeler** or "Gatekeeper" for a simple Trend Following strategy.

- 1) **Base Signal**: A simple Moving Average crossover suggests a Long trade.
- 2) **Filter**: The XGBoost model analyzes the market context (RSI, Volatility, Volume). It outputs a probability score $[0, 1]$ indicating the likelihood that the Base Signal will result in a profit.
- 3) **Decision**: We only execute the trade if the XGBoost probability exceeds a strict threshold ($\tau > 0.6$).

The feature vector F_t includes Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), normalized volume, and volatility ratios.

Algorithm 1 XGBoost Momentum Gatekeeper

Require: P_t (Price), F_t (Feature Vector), τ (Threshold)

- 1: Calculate trend $MA_{trend} = \text{Mean}(P_{t-w:t})$
 - 2: Generate base signal $S_{base} = \mathbb{I}(P_t > MA_{trend})$
 - 3: **if** $S_{base} == 1$ **then**
 - 4: *// Ask XGBoost: Is this a fake breakout?*
 - 5: $Prob_{up} \leftarrow \text{XGBoost.predict_proba}(F_t)$
 - 6: **if** $Prob_{up} > \tau$ **then**
 - 7: $Signal_t \leftarrow 1$ {High Confidence: Enter Long}
 - 8: **else**
 - 9: $Signal_t \leftarrow 0$ {Low Confidence: Filter Trade}
 - 10: **end if**
 - 11: **else**
 - 12: $Signal_t \leftarrow 0$
 - 13: **end if**
 - 14: **return** $Signal_t$
-

Reproducibility Note: The complete source code, including the data pipeline, model architectures, and backtesting engine, is available at our public repository: <https://github.com/Alextz307/Stock-Prediction-Methods>.

III. DESIGN OF EXPERIMENTS

A. Dataset Description

The experimental universe consists of diverse assets to test robustness across sectors. Data is sourced via Yahoo Finance.

- **Assets**: SPY (Index), AAPL (Tech), NVDA (High Beta), KO (Low Beta).
- **Training Set**: 2020-05-18 to 2024-05-09 (Model Fitting).
- **Validation Set**: 2024-05-09 to 2025-05-09 (Hyperparameter Tuning).
- **Test Set (OOS)**: 2025-05-09 to Present (Final Performance).

B. Bayesian Optimization with TPE

We utilize the **Tree-structured Parzen Estimator (TPE)** algorithm via the Optuna framework for hyperparameter optimization. TPE significantly outperforms Grid Search and Random Search by building a probabilistic model of the objective function.

1) *Why TPE?*: Standard Grid Search suffers from the "Curse of Dimensionality," where adding parameters exponentially increases computational cost. TPE is a **Sequential Model-Based Optimization (SMBO)** method. Instead of testing random points, it "learns" from previous trials to predict where the best hyperparameters are likely to be found.

2) *How TPE Works (The Dual Density Approach)*: Most Bayesian optimization methods (like Gaussian Processes) model the probability of a score given the parameters, $P(y|x)$. TPE flips this. It models the probability of the parameters given the score, $P(x|y)$.

It splits all observed trials into two groups based on a threshold y^* (typically the top 15% of best scores):

- $l(x)$: The distribution of parameters that yielded **Good** results.

- $g(x)$: The distribution of parameters that yielded **Bad** results.

The algorithm then proposes the next set of hyperparameters by choosing x that maximizes the ratio:

$$\text{Expected Improvement} \propto \frac{l(x)}{g(x)} \quad (5)$$

Intuitively, TPE picks parameters that are highly likely to be found in the "Good" distribution $l(x)$ and highly unlikely to be found in the "Bad" distribution $g(x)$. This allows the search to quickly converge on optimal regions, saving significant computational time compared to brute-force methods.

TABLE I: Hyperparameter Search Space (TPE Optimization)

Strategy	Parameter	Range
VolTarget	Target Volatility	[0.05, 0.40]
	Trend Window	[0, 252]
	MA Type	{SMA, EMA}
Adaptive Bollinger	BB Window	[10, 50]
	Multiplier (k)	[1.0, 3.0]
	Trend Window	[50, 252]
	MA Type	{SMA, EMA}
XGBoost Gatekeeper	Prob Threshold (τ)	[0.51, 0.70]
	Trend Window (MA)	[10, 100]

C. Implementation Details

The system is implemented in Python 3.10.

- **VolTarget Strategy:** Leverages the Hybrid GARCH-LSTM model to dynamically size positions ($\text{Size} = \text{TargetVol} / \hat{\sigma}_{\text{Hybrid}}$). **Crucially, we incorporate a Regime Filter:** if the asset price is below its long-term moving average (defined by the *Trend Window* hyperparameter), the system identifies a bearish regime and overrides the position size to zero (Cash), regardless of the volatility forecast.
- **MomentumXGB Strategy:** Uses the classifier to validate breakouts.
- **Transaction Costs:** Fixed at 10 basis points (0.10%) per trade to simulate slippage and fees.

IV. RESULTS AND DISCUSSION

A. Quantitative Performance

The strategies were evaluated on a diverse basket of assets covering large-cap tech, high-beta growth, and defensive sectors. Table II details the performance metrics. We utilize the **Sharpe Ratio** as our primary metric for evaluation.

Definition 1 (Sharpe Ratio): The Sharpe Ratio measures the excess return per unit of deviation (risk). It is calculated as $S_a = \frac{E[R_a - R_f]}{\sigma_a}$, where R_a is the asset return, R_f is the risk-free rate, and σ_a is the standard deviation of the returns. A higher Sharpe Ratio indicates superior risk-adjusted performance, implying the strategy is generating returns through skill rather than by taking on excessive exposure.

B. Discussion and Analysis

1) *The "Sniper" Effect: Precision Over Frequency:* The most striking result is observed in **AAPL**, where the MomentumXGB strategy achieved a Sharpe Ratio of 1.62 compared to the Benchmark's 0.53. Crucially, it did so with a Win Rate of only 2.4% and a Maximum Drawdown of just -1.9% (vs -30.2% for Buy-and-Hold).

This confirms the "Gatekeeper" hypothesis: the XGBoost model effectively filtered out the vast majority of noise, executing trades only when probability thresholds were extremely high. While the total return (7.1%) was lower than the benchmark (12.5%), the risk profile was transformed from a volatile equity to a bond-like stability. As seen in Fig. 1, the strategy (orange line) remains flat during the severe drawdowns of the benchmark (blue line).

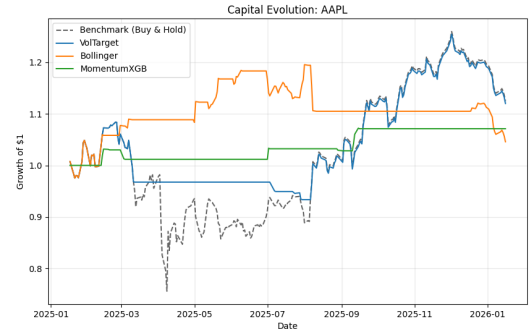


Fig. 1: AAPL: Comparison of MomentumXGB Strategy vs. Buy-and-Hold Benchmark. Note the capital preservation during market downturns.

2) *Volatility Targeting in High-Beta Regimes:* For assets with extreme variance like **TSLA** and **GOOG**, the Volatility Targeting framework outperformed significantly.

- In **TSLA**, the strategy doubled the Sharpe Ratio (0.82 vs 0.40) and cut the Max Drawdown by more than half (-23.2% vs -48.0%).
- In **GOOG**, it achieved the "Holy Grail" of trading: higher total returns (78.7%) with lower drawdown (-12.2%).

The Hybrid GARCH-LSTM model successfully anticipated volatility expansions, allowing the algorithm to deleverage before major corrections. Furthermore, the inclusion of the **Regime Filter** ensured that the system did not attempt to "catch falling knives" during sustained downtrends, preserving capital until the trend realigned. Fig. 2 illustrates this protective mechanism.

3) *Limitations and Mean Reversion:* The results for **NVDA** and **MSFT** highlight the limitations of risk-averse strategies during strong unilateral bull markets. In NVDA, the Benchmark outperformed (Sharpe 0.93) because the VolTarget mechanism repeatedly reduced exposure during high-volatility upside rallies, missing out on "good volatility."

Interestingly, the traditional **Bollinger Band** strategy outperformed in the energy sector (**XOM**, Sharpe 1.64), suggest-

TABLE II: Comparative Performance Metrics (Out-of-Sample)

Ticker	Strategy	Sharpe Ratio	Total Return	Max Drawdown	Win Rate
SPY	Benchmark	0.97	18.3%	-18.8%	57.0%
	VolTarget	1.34	14.2%	-8.5%	45.8%
	MomentumXGB	0.83	6.0%	-6.0%	27.1%
AAPL	Benchmark	0.53	12.5%	-30.2%	52.2%
	MomentumXGB	1.62	7.1%	-1.9%	2.4%
GOOG	Benchmark	1.85	70.6%	-29.4%	54.6%
	VolTarget	2.44	78.7%	-12.2%	42.6%
TSLA	Benchmark	0.40	5.7%	-48.0%	49.8%
	VolTarget	0.82	27.4%	-23.2%	26.7%
XOM	Benchmark	0.91	21.0%	-16.1%	55.8%
	Bollinger	1.64	22.3%	-10.7%	16.7%
NVDA	Benchmark	0.93	39.5%	-35.9%	53.8%
	VolTarget	0.57	15.1%	-27.1%	42.6%

Note: Only select assets and top-performing strategies shown for brevity. Full results available in the repository.

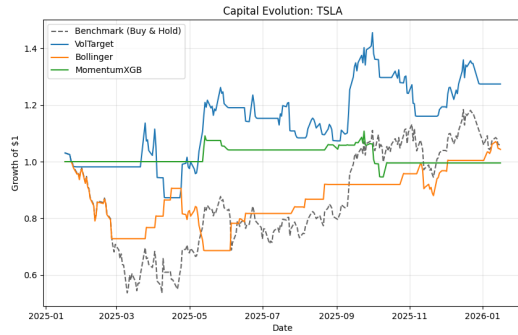


Fig. 2: TSLA: Volatility Targeting Strategy vs. Benchmark. The strategy mitigates the massive -48% drawdown observed in the underlying asset.

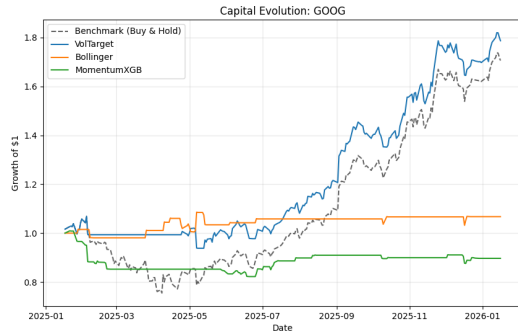


Fig. 3: GOOG: Volatility Targeting Strategy vs. Benchmark. Demonstrates alpha generation with superior risk-adjusted returns.

ing that mean-reversion dynamics remain superior to trend-following in cyclical, lower-beta industries.

V. CONCLUSION

This paper presented a regime-adaptive trading framework integrating econometric theory with modern machine learning. The results validate that: (1) Hybrid volatility models outperform pure linear models in risk estimation, and (2) ML-based gatekeeping significantly improves the Sharpe ratio of trend-following strategies. Future work will explore replacing the LSTM with Transformer architectures to better capture long-range dependencies in volatility structures.

REFERENCES

- [1] M. B. Garman and M. J. Klass, "On the estimation of security price volatilities from historical data," *Journal of Business*, vol. 53, no. 1, pp. 67–78, 1980. [Online]. Available: <http://www.jstor.org/stable/2352358>
- [2] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," *Journal of Econometrics*, vol. 31, no. 3, pp. 307–327, 1986. [Online]. Available: [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1)
- [3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [5] H. Y. Kim and C. H. Won, "Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models," *Expert Systems with Applications*, vol. 103, pp. 25–37, 2018. [Online]. Available: <https://doi.org/10.1016/j.eswa.2018.03.002>
- [6] M. Lopez de Prado, *Advances in Financial Machine Learning*. Hoboken, NJ, USA: Wiley, 2018.
- [7] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011. [Online]. Available: <https://proceedings.neurips.cc/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf>